

差错控制（一）

陈巍

2025年秋

本讲内容摘要

- ① 差错控制的目的与主要方法
- ② 码本的检错和纠错能力
- ③ 典型检错算法与“停-等”重传

要求重点掌握：

- ① 了解 and 对比三种典型差错控制体制
- ② 由最小码距计算纠错位数和检错位数
- ③ 会用典型检错算法进行检错判断，并针对停等ARQ，分析其吞吐量和差错概率

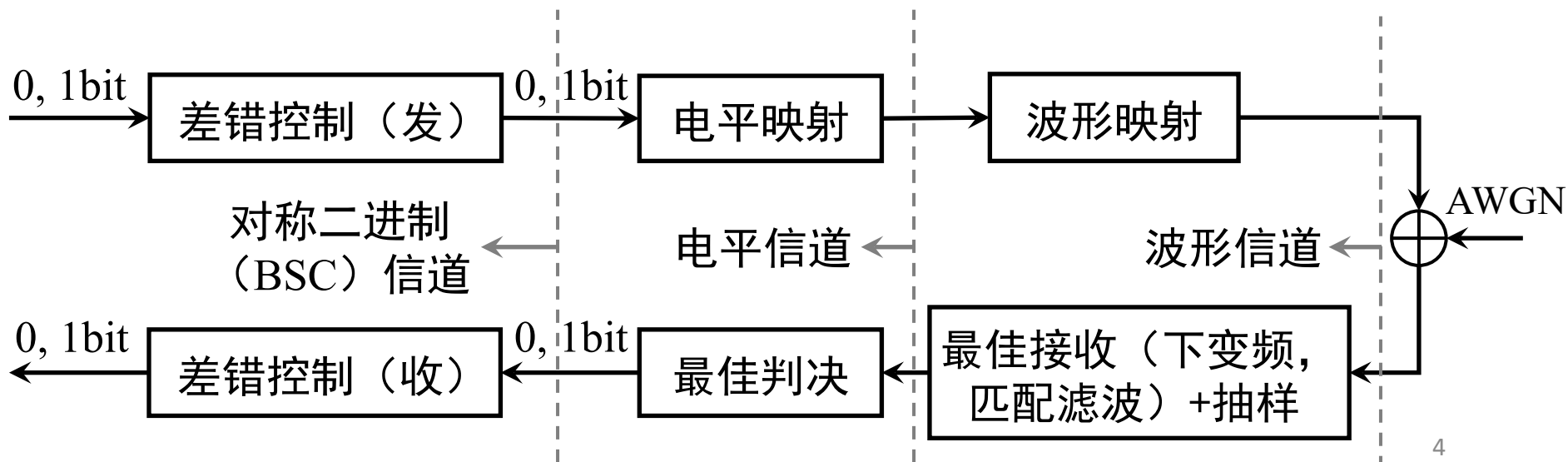
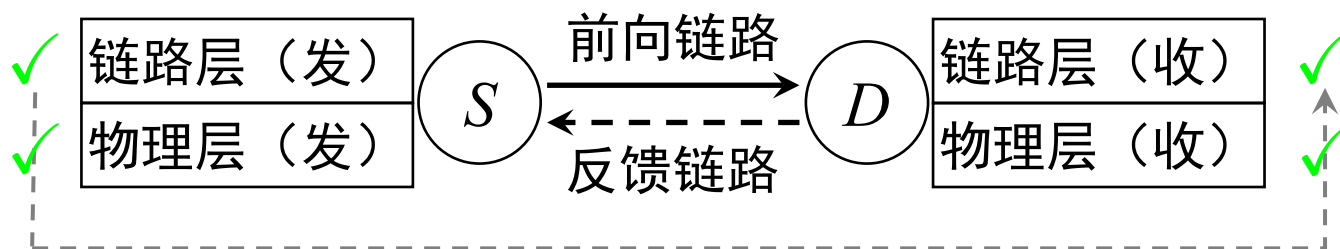
差错控制的目的与位置（一）

为什么要专门讨论差错控制？

- ① 不同的网络业务有不同的可靠性要求；（话音，图像，卫星测控，远程手术，金融数据，……）
- ② 数字调制中，仅当信噪比 $\frac{E_b}{n_0} \rightarrow +\infty$ 时，误bit率才能降为0！且单纯依靠提升功率来降低 P_b 的方法不实用；
- ③ 网络中有其他破坏可靠性的因素（丢包，干扰，DDOS攻击，自然灾害，……）

差错控制的目的与位置（二）

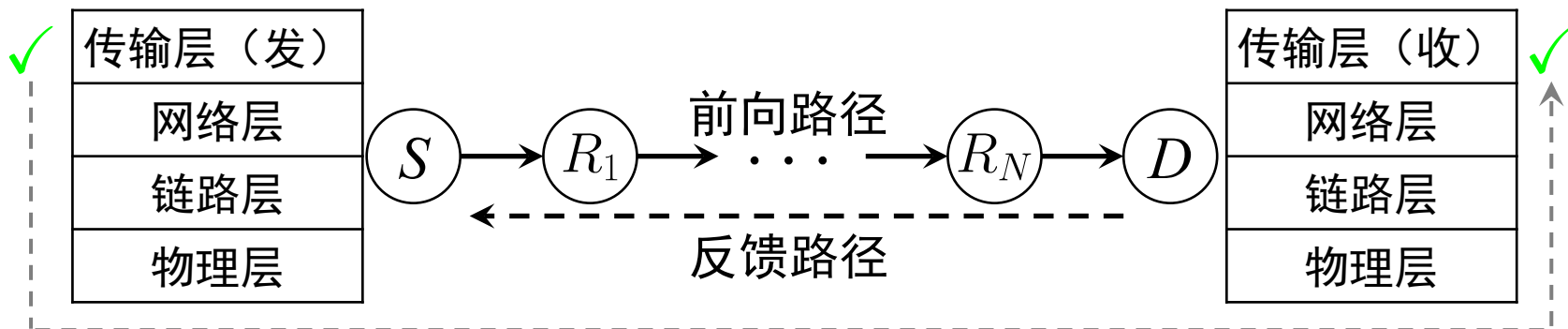
差错控制的位置 I —— 物理层，链路层
点到点，单跳



差错控制的目的与位置（三）

差错控制的位置 II —— 传输层

端到端，多跳甚至多路径

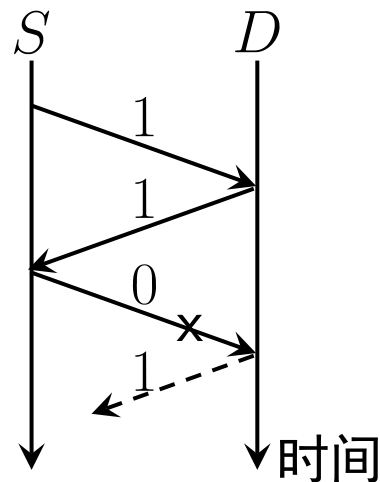
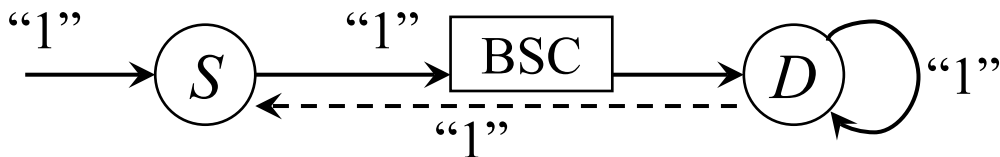


目的：针对网络中的各类不可靠因素提升端到端的传输可靠性。

方法：中间不处理（默认不可靠），收端确认已正确传输的包，发端重传“传错”或“传丢”的包。

差错控制的主要方法（一）

（一）反馈原信息进行确认



D 把从 S 处收到的bit原样发回给 S ,

S 判断 { ①与所发一致，则发下一个bit分组
②与所发不一致，则重发

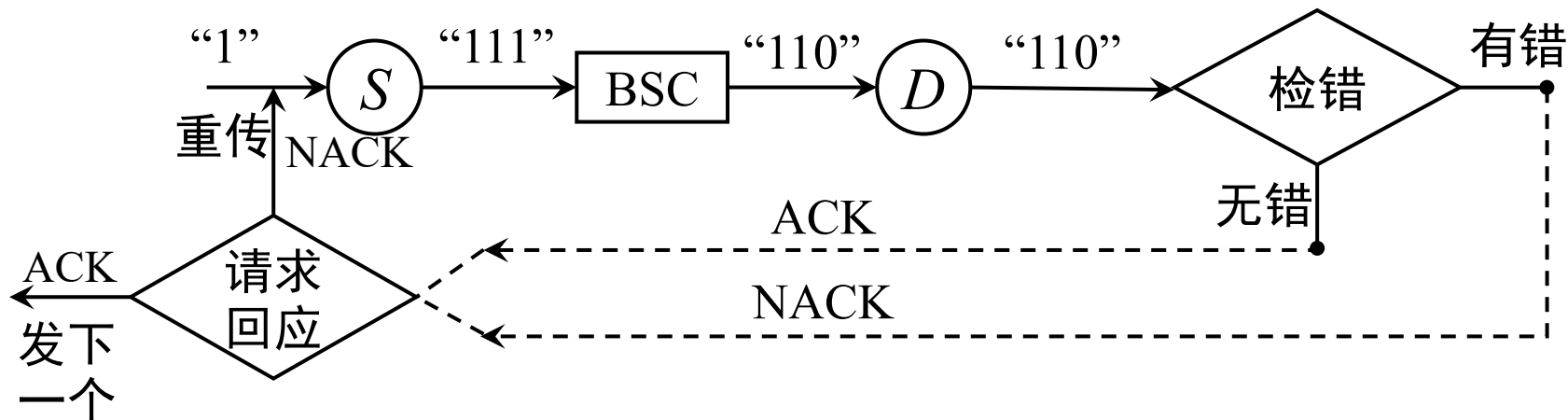
许多操作手的
确认方式！

问题 1：是否可确保绝对可靠？（否，除非反馈绝对可靠）

2：有什么缺点？（效率太低！）

差错控制的主要方法（二）

（二）检错重发（Automatic Repeat reQuest, ARQ）



D 对准备发送的信息bit，添加冗余，用以检错

$\left. \begin{array}{l} \text{"1"} \rightarrow \text{"111"} \\ \text{"0"} \rightarrow \text{"000"} \end{array} \right\} \text{代价：降低了通信效率，速率}$

除非收到 "111" 或 "000"，都重发。

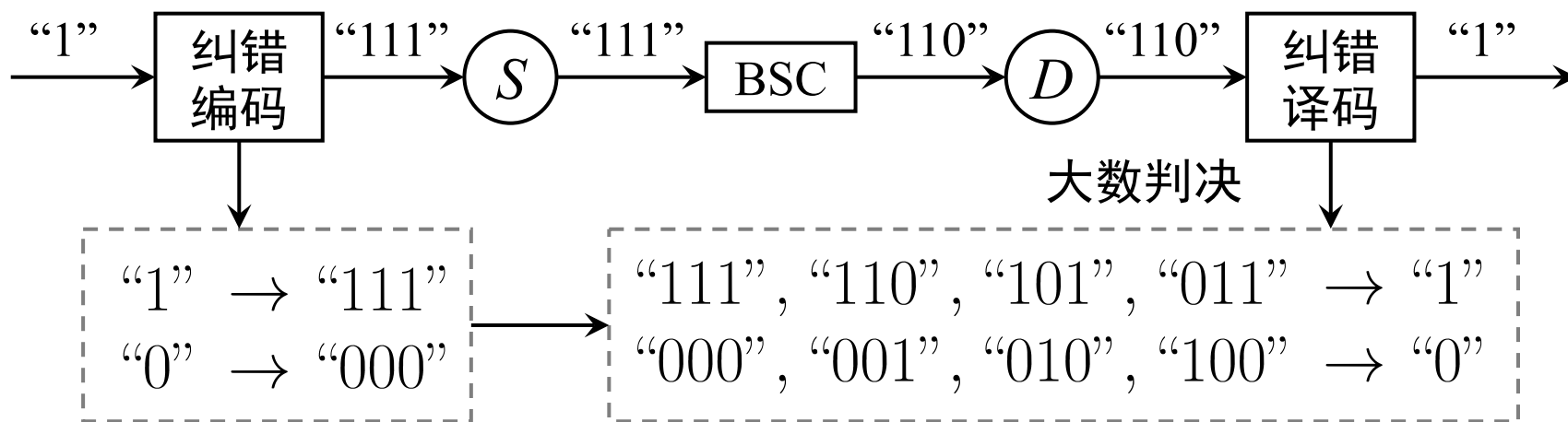
差错控制的主要方法（三）

（三）前向纠错（Forward Error Correction, FEC）

动机：有一些通信系统，① 无反馈链路（电视）

② 不能容忍重传延时，则无法使用ARQ

解决方案：“相信大多数！”



以上方法，都难以确保绝对可靠！

码本的检错和纠错能力（一）

定义：信息bit串映射的结果，称为码字；码字的集合，称码字集合；码字集合连同映射关系，称为码本。

例：“1” $\rightarrow$ “111” “0” $\rightarrow$ “000”

对比上两页中的ARQ和FEC对接收bit串的处理

ARQ： 110, 101, 011, 100, 010, 001 这六种被认

为是有错 $\longrightarrow$ 检2位错

FEC： 110, 101, 011, 被认为是“1”
100, 010, 001, 被认为是“0”

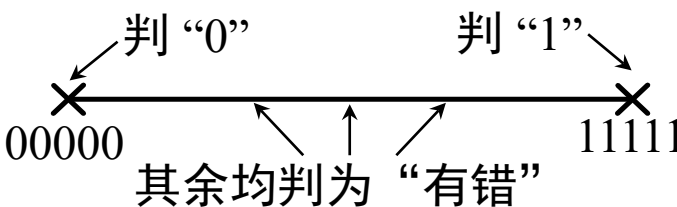
纠1位错

对同一个串
的处理不同，
即纠、检错
不完全兼容

码本的检错和纠错能力（二）

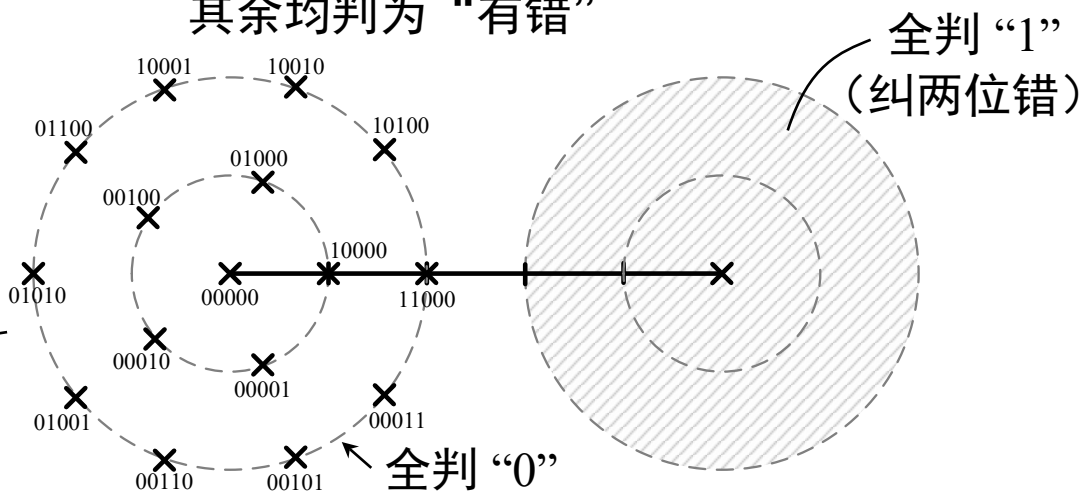
再考虑一个例子，以寻求一般结论：
“1” → “11111”
“0” → “00000”

方案一（纯检错）：



(检4位错)
全错则检不出来

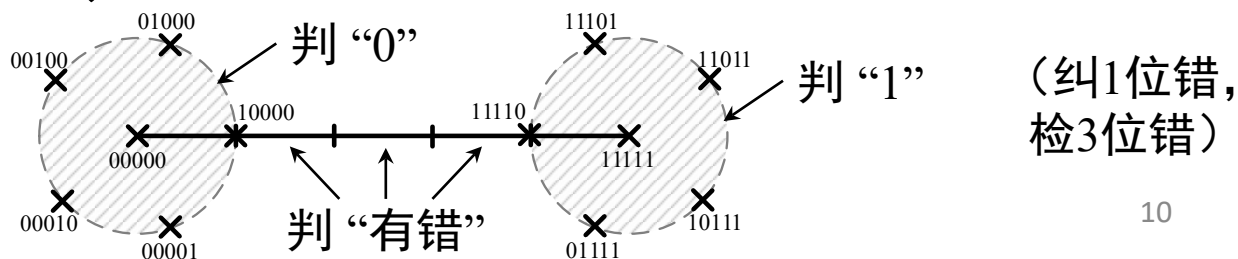
方案二（纯纠错）：



相异bit越少，
说明越像

全判“1”
(纠两位错)

方案三（既纠错，又检错）：

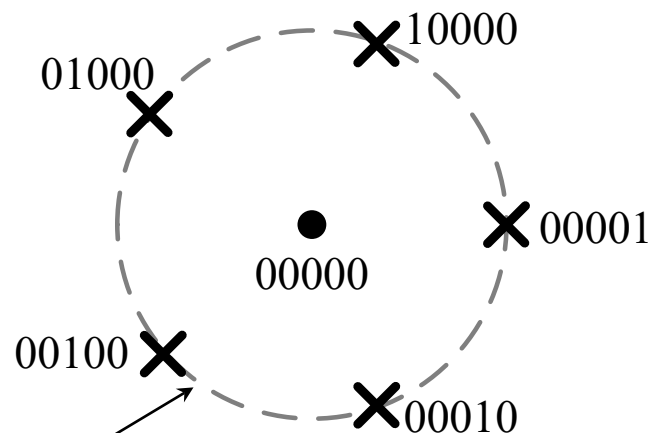


码本的检错和纠错能力（三）

从以上两个例子中可以看出：① 两个bit串，其相异bit个数体现了其“相似”的程度。② 两个码字，其相异的bit个数决定了其检错或纠错的能力！

定义：两个bit串（或码字）对应位之间相异的bit个数称为汉明距离（Hamming Distance）

例： $d(000, 111) = 3$
 $d(000, 101) = 2$
 $d(00000, 11111) = 5$
 $d(00000, 10000) = 1$



汉明距1的球
 $\{c : d(c, 00000) = 1\}$

码本的检错和纠错能力（四）

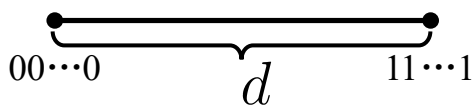
推广 $d = 3, d = 5$ 的情况，到一般的 d ，即考虑码本的纠错、

$$\begin{array}{ccc} \text{“0”} \rightarrow \underbrace{000 \cdots 0}_{d \text{ 位}} & & \text{“1”} \rightarrow \underbrace{111 \cdots 1}_{d \text{ 位}} \end{array}$$

检错能力，记 t 为纠错位数， e 为检错位数 ($e \geq t$)

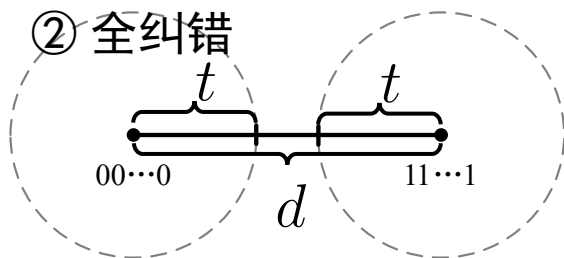
归纳：

① 全检错



——> 只要不错成全1，都能检出“野生”bit串，此时 $e = d - 1$
无法纠错，则 $t = 0$

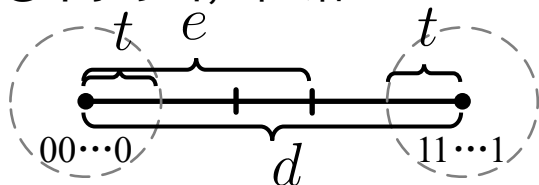
② 全纠错



(默认 d 为奇数) (若 d 为偶，则 $t = \lfloor \frac{d-1}{2} \rfloor$)

——> 距离 $00 \dots 0$ 近的全判 0，距离 $11 \dots 1$ 近的全判 1。此时， $2t = d - 1$ ，而任一bit串一定被一圆“捕获”，不存在“野生”bit串，故不存在检出为错但又无法纠的。

③ 同时纠，检错



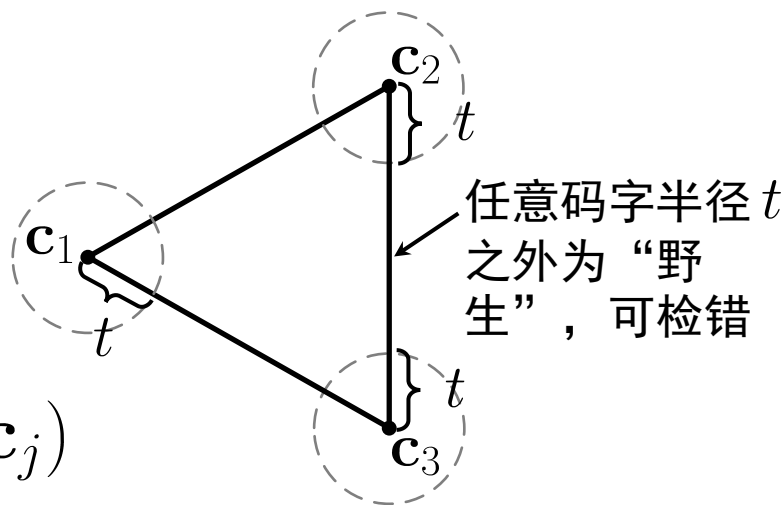
两个半径为 t 的圆之外的均为“野生”bit串，圆之内的被“捕获”而纠错，故 $t + e + 1 = d$

码本的检错和纠错能力（五）

刚才，我们推广到汉明距为一般 d 的全0，全1串组成的码本。
现在，再推广到一般码本。

定义：码字集合中，任意两个码字之间的汉明距离的最小值，称

最小汉明距离 $d_{\min} = \min_{i,j} d(\mathbf{c}_i, \mathbf{c}_j)$



定理：给定码本，其纠错位数 t 和检错位数 e 满足

$$t + e + \textcircled{1} \leq d_{\min} \quad (\text{码字之间是离散的})$$

证明思路： t, e 对任意码字都要满足，考虑两个最近的 $\mathbf{c}_1$ 和 $\mathbf{c}_2$ ，
距离 $\mathbf{c}_1$ 有 $d_{\min} - t$ 远的，必有一个被 $\mathbf{c}_2$ 捕获，故最多纠 $e = d_{\min} - t - 1$ 位

推论：① $2t + 1 \leq d_{\min}$ ② $e + 1 \leq d_{\min}$

典型的检错算法（一）

有了差错后，可能的bit串的个数，随其长度指数增长，依靠与许用码字c算距离来判断“野生”，复杂度太高。工程上需要简洁的检错方法（少量代数运算，且易于电路实现）

本课程差错控制编译码的代数运算均在 $GF(2)$ 上

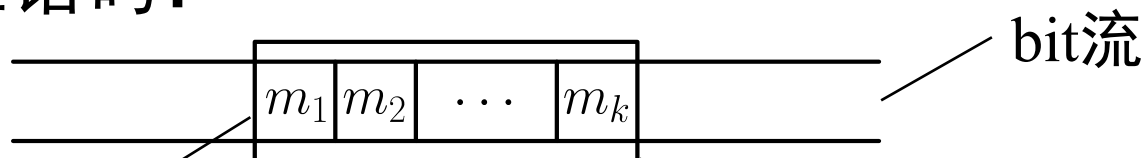
加法： $0 + 0 = 0$ $0 + 1 = 1 + 0 = 1$ $1 + 1 = 0$ （模2和）
XOR, $\oplus$

乘法： $0 \cdot 0 = 0$ $0 \cdot 1 = 1 \cdot 0 = 0$ $1 \cdot 1 = 1$

$GF(2)$ 上也可以定义矢量，矩阵的运算，数的运算法则同上。运算的对应位置，可交换性等，同 $\mathbb{R}$ 上的矢量矩阵运算。

典型的检错算法（二）

奇偶校验检错码：



这里，用 m_i 表示信息bit (message)

切出待编码的信息bit分组
一组含 k 个信息bit

编码方式： $c_i = \begin{cases} m_i & i = 1, \dots, k \\ \sum_{j=1}^k m_j & i = k+1 \end{cases}$ ，矩阵形式 $\mathbf{c} = \mathbf{m} \begin{bmatrix} 1 & 0 & \dots & 0 & 1 \\ 0 & 1 & \dots & 0 & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & 1 & 1 \end{bmatrix}$
(分组，线性)

解码方式：注意到 $\sum_{i=1}^{k+1} c_i = m_1 + m_2 + \dots + m_k + \sum_{j=1}^k m_j$ } 必有偶数个
 $= (m_1 + m_1) + (m_2 + m_2) + \dots + (m_k + m_k)$ "1"，故符合
 $= 0 + 0 + \dots + 0 = 0.$

故 $\sum_{i=1}^{k+1} c_i = 0$ (或 $\mathbf{c} \cdot \mathbf{1} = 0$) 时，判为无错，否则报错，
请求重发

典型的检错算法（三）

奇偶校验的检错能力： $e = 1$ ， $(t = 0)$

原因： c 中错两个bit及任意偶数个bit时 c_1 仍为0，无法检错。

性质： $d_{\min} = 2$ ，（由此必有 $t = 0$ ）

证明：对 c ，翻转其中任意两个bit后，得到的 c' 仍满足 $c'_1 = 0$

说明 c' 也是“合法”码字，故 $d(c, c') = 2$ 。

漏检概率（ p_m ）为一个检错码发生差错，但未能检出的概

率。对奇偶校验码

错2位

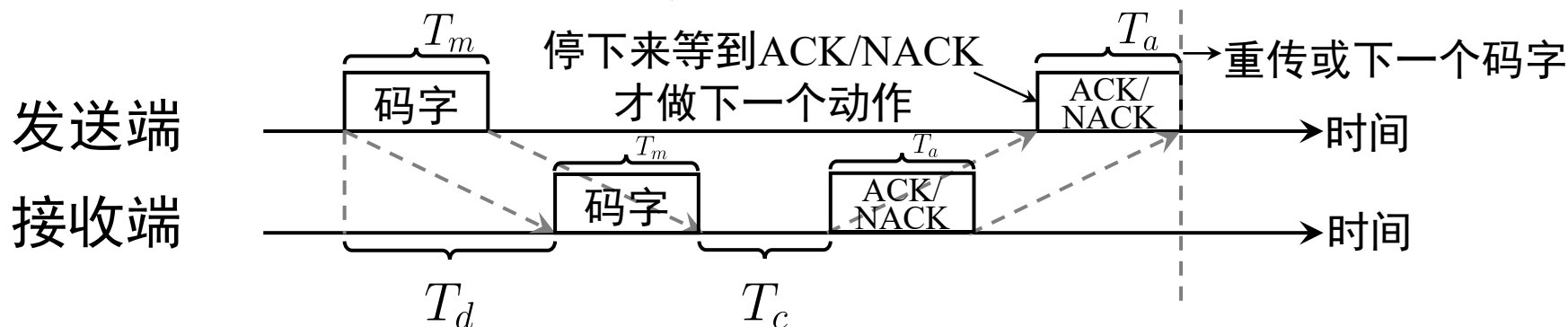
错4位

错 $2 \left\lfloor \frac{k+1}{2} \right\rfloor$

$$\begin{aligned} p_m &= \binom{k+1}{2} P_b^2 (1 - P_b)^{k-1} + \binom{k+1}{4} P_b^4 (1 - P_b)^{k-3} + \cdots + \binom{k+1}{2 \left\lfloor \frac{k+1}{2} \right\rfloor} P_b^{2 \left\lfloor \frac{k+1}{2} \right\rfloor} (1 - P_b)^{k+1-2 \left\lfloor \frac{k+1}{2} \right\rfloor} \\ &= \sum_{i=1}^{\left\lfloor \frac{k+1}{2} \right\rfloor} \binom{k+1}{2i} P_b^{2i} (1 - P_b)^{k+1-2i} \end{aligned}$$

“停-等” 重传机制（一）

有了检错能力，我们就可以设计分析重传协议，先讨论一种最简单的“停-等”机制，但不忽略其物理细节



码字： k 个信息bit，映射（编码）成 n 个编码后的bit（码字长度，对奇偶校验码 $n = k + 1$ ）

T_m ： 传输一个码字的时间（传完才能译码）， $T_m = \frac{n}{R_b}$

T_d ： 传播延时（对无线电， $T_d = \frac{\text{传播距离}}{\text{光速}}$ ）

T_c ： 译码的计算时间

T_a ： 传输一个ACK/NACK的时间， $T_a = \frac{\text{ACK/NACK串的比特计数}}{R_b}$

“停-等” 重传机制（二）

我们感兴趣停等ARQ的效率，在一个轮次中真正用于传输码字（注意，还不是消息bit）的时间为 T_m ，额外的时间开销为：

$$T_{dca} = 2T_d + T_c + T_a$$

(一来一回)

即使一轮传输成功（什么叫成功？）其效率（有效码字传输时间在一轮中的占比）为

$$\xi' = \frac{T_m}{T_m + T_{dca}}$$

问题1：平均几轮才能成功？（收到ACK，开始传下一个码字？）

问题2：如何定义成功？（无错，或，有错但检不出来）

“停-等” 重传机制（三）

成功（ACK）概率 $p_{suc} = \underbrace{(1 - P_b)^n}_{n \text{ 个 bit 都对}} + \underbrace{p_m}_{\text{有错检不出来}} \longrightarrow$ 不成功（NACK）概率 $1 - p_{suc}$

在第 i 轮成功的概率为 $\underbrace{(1 - p_{suc})^{i-1}}_{\text{前}(i-1)\text{次失败}} \underbrace{p_{suc}}_{\text{第 } i \text{ 次成功}} \longrightarrow$ 几何分布（抛硬币直到正面向上）

于是收到ACK的平均轮次数为 $\frac{1}{p_{suc}}$

因此，要收到ACK需等待的时间为 $(T_m + T_{dca})/p_{suc}$

故考虑随机误码引起重传，其效率为 $\xi = \xi' p_{suc} = \frac{T_m p_{suc}}{T_m + T_{dca}}$

最后，注意到码字的 n 个bit中只有 k 个信息（消息）bit，

故上层数据bit流体验到的速率为

$$r = \frac{k}{n} \cdot \xi R_b = \frac{k p_{suc}}{n} \cdot \frac{T_m R_b}{T_m + T_{dca}} = \frac{k [(1 - P_b)^n + p_m]}{n} \cdot \frac{T_m R_b}{T_m + T_{dca}}.$$

（对于奇偶校验，可将 $n = k + 1$ 和P16的 p_m 自行带入后获得其对上层的服务速率）

谢谢！

Thanks!